

Registrazione di mappe catastali storiche su cartografia moderna

Progetto d'esame di Computer Vision — Università di Pavia

Caso di studio: Comune di Varazze (SV), foglio 49, Originale di Impianto dell'Agenzia delle Entrate.

Le tabelle di questo documento sono generate automaticamente dai risultati sperimentali raccolti in `results/runs.csv`, non trascritte a mano: se un numero cambia, cambia perché è cambiato l'esperimento.

1. Il problema

Un **Originale di Impianto** è il foglio di mappa disegnato a mano all'atto della formazione del catasto terreni, nella prima metà del Novecento: un disegno al tratto su carta, con i confini delle particelle, i corsi d'acqua, le strade, numeri scritti a penna, e un secolo di ingiallimento e macchie sopra.

Registrarlo su una cartografia moderna significa trovare la trasformazione geometrica che porta i pixel della scansione storica a coincidere con le coordinate della mappa attuale. Il problema è più difficile di quanto sembri, per una ragione precisa: le tecniche standard per riconoscere punti corrispondenti fra due immagini nascono per le fotografie, dove ogni zona dell'immagine ha un aspetto diverso dalle altre. Qui invece **entrambe le immagini sono disegni al tratto**, quasi senza texture, e per di più con un aspetto completamente diverso — inchiostro sbiadito da un lato, linee vettoriali nitide dall'altro. È un caso sfavorevole quasi apposta.

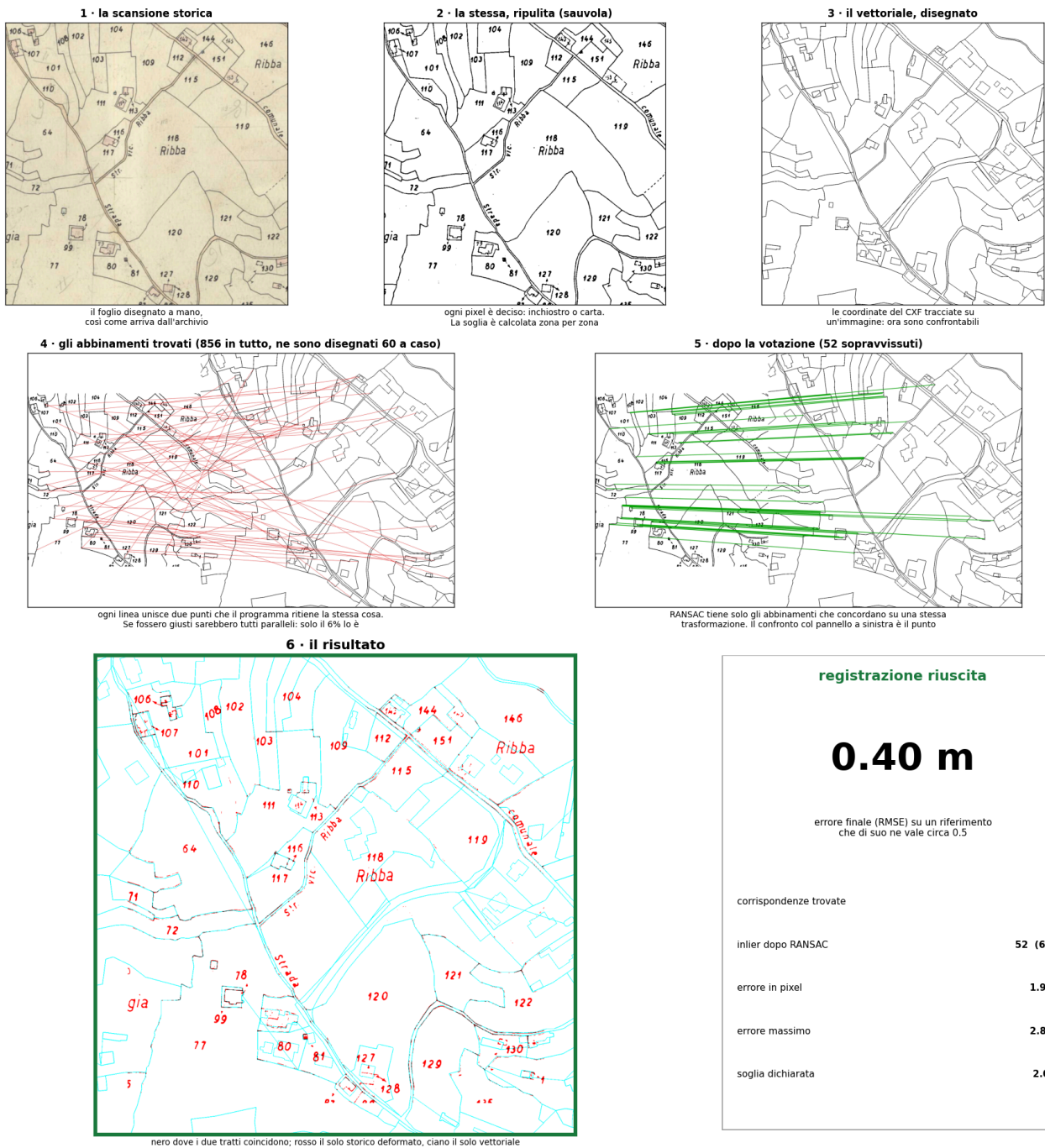
Il progetto affronta il problema in due parti, secondo la traccia d'esame approvata dal docente:

- **A — approccio classico:** pulizia dell'immagine, ricerca di punti corrispondenti con i metodi tradizionali (SIFT, ORB) e stima della trasformazione per votazione (RANSAC), cercando di spingere questi metodi al massimo tramite una preparazione accurata del segnale.
- **B — confronto:** la stessa pipeline, con un solo componente sostituito da un metodo di deep learning (LoFTR), per misurare se una rete pre-addestrata colma il divario fra i due stili di disegno meglio dei metodi classici.

Il caso di studio è il foglio 49 del Comune di Varazze, scelto perché l'Agenzia delle Entrate fornisce, insieme alla scansione, anche la sua posizione geografica esatta e la cartografia vettoriale aggiornata dello stesso foglio. Questo permette — ed è il punto metodologico su cui si regge l'intero lavoro — di **misurare l'errore in metri senza dover indovinare la risposta giusta** (§3).

2. Come funziona il sistema

Una registrazione dall'inizio alla fine — ritaglio «ribba», ORB · sauvola · similarity



Il programma riceve due immagini della stessa zona — la scansione storica e un raster ricavato dalla mappa vettoriale moderna — e restituisce la trasformazione che le sovrappone. Il procedimento, illustrato in figura, si articola in cinque passi.

Pulizia. Ogni pixel viene classificato come inchiostro o come carta, eliminando le variazioni di colore della carta stessa (§4).

Ricerca di punti riconoscibili. Il programma cerca, in entrambe le immagini, punti "distintivi" — un incrocio di confini, l'angolo di un fabbricato — e per ciascuno calcola un'impronta numerica che ne descrive l'aspetto locale. È il ruolo dei descrittori SIFT e ORB (§4).

Abbinamento. Ogni punto della prima immagine viene accoppiato con il punto della seconda che ha l'impronta più simile. Su un disegno al tratto molti incroci si somigliano, e la maggior parte di questi abbinamenti è sbagliata: nell'esempio in figura solo il 6% è corretto.

Votazione (RANSAC). Il programma estrae a caso due abbinamenti alla volta, calcola quale trasformazione implicherebbero, e conta quanti altri abbinamenti sarebbero d'accordo con quella stessa trasformazione. Ripetuto migliaia di volte, il procedimento tiene la trasformazione più votata. È il passo che rende il metodo utilizzabile anche quando, come qui, la stragrande maggioranza degli abbinamenti è rumore.

Verifica. La trasformazione vincente viene applicata all'immagine storica, che così si sovrappone al vettoriale: dove i due disegni coincidono l'overlay appare nero, dove no si vede una frangia colorata.

Come si giudica il risultato. Sia la scansione storica sia il file vettoriale portano con sé un *world file(jgw)*: sei numeri che indicano a quali coordinate reali corrisponde ogni pixel. Da questi numeri si calcola per via puramente algebrica la trasformazione **esatta**, senza dover annotare a mano un solo punto di controllo (§3). Il programma che stima la trasformazione non vede mai questi numeri — li usa solo il codice di valutazione, separato — così il confronto fra stima e verità resta un test indipendente e non un esercizio con le soluzioni già viste.

3. I dati e la verità di riferimento

Dal servizio dell'Agenzia delle Entrate si ottengono, per il foglio 49 di Varazze: la scansione dell'Originale di Impianto (8489 × 5648 pixel), il suo world file(formato JGW), e un file vettoriale (formato CXF) con la cartografia catastale **vigente**, nello stesso sistema di riferimento della scansione — un dettaglio non scontato, perché il servizio distribuisce per lo stesso foglio anche una seconda variante vettoriale in un sistema diverso, facile da confondere con la prima e già causa in passato di un pomeriggio perso dietro un allineamento apparentemente senza senso.

Il file CXF è un formato testuale elementare, letto da un parser scritto da zero in una trentina di righe. Un singolo campo di formato, presente solo in 33 record su 871, se ignorato sposta la lettura delle coordinate di qualche posizione e fa esplodere l'estensione del foglio da 1.6 a 31 chilometri: il programma verifica automaticamente, dopo ogni lettura, che ogni coordinata ricada dentro l'area attesa, proprio per intercettare questo tipo di errore silenzioso.

Il foglio intero è troppo grande per essere utile allo scopo dell'esame, per cui si lavora su cinque **ritagli** di circa 1000-1200 pixel di lato, scelti nella fascia collinare dove la mappa vettoriale copre per intero ciò che è disegnato nell'impianto (sono state escluse le zone di confine

amministrativo, vuote in entrambe le rappresentazioni, e il centro urbano, dove il catasto terreni non ha competenza e quindi il vettoriale non ha nulla da confrontare).

3.1 La ground truth: esatta e gratuita

Questo è il punto metodologico su cui poggia tutto il resto.

Il world file `L675_004900.jgw` contiene sei numeri che definiscono una trasformazione affine da pixel a coordinate: la dimensione del pixel sui due assi — **0.254453 m**, cioè la scala 1:2000 — due termini di rotazione, qui nulli, e le coordinate del centro del primo pixel, (−31480.044315, −11278.758056). Da questi sei numeri la posizione sul terreno di qualunque pixel si ottiene con due moltiplicazioni e due somme.

Rasterizzando il CXF su una griglia di cui conosciamo, allo stesso modo, il legame fra pixel e coordinate, si ottengono due trasformazioni note, e la trasformazione di riferimento è la loro composizione:

$$H_{true} = \frac{W_{storico}}{W_{moderno}}$$

Non è stato annotato un solo punto di controllo a mano. La ground truth è analitica, esatta per costruzione, e `evaluate.py` la usa per produrre l'RMSE in metri su una griglia regolare di checkpoint nell'immagine storica (10×10 punti, bordi esclusi):

$$e_i = \|H_{est} \cdot p_i - H_{true} \cdot p_i\|_2$$
$$RMSE_m = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2} \cdot r_{dest}$$

dove $N = 100$ sono i checkpoint e r_{dest} è la risoluzione in metri per pixel della griglia di arrivo: 0.254453 m/px in E1, dove la coppia è il ritaglio contro se stesso trasformato, e 0.20 m/px in E2, dove l'arrivo è il raster del vettoriale (§9.1).

cioè: si applicano entrambe le trasformazioni — quella stimata e quella vera — agli stessi 100 punti, si misura di quanto le due risposte divergono, e si converte in metri. È l'unico numero su cui si giudica l'intera pipeline: ogni tabella dei capitoli successivi è, in ultima analisi, un'aggregazione di questa formula su configurazioni diverse.

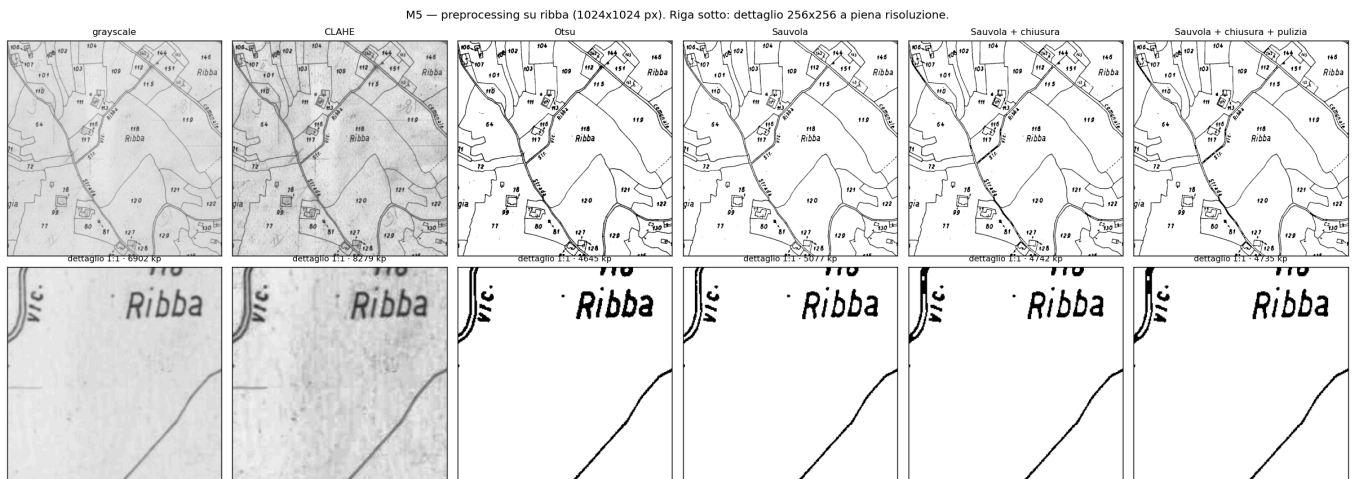
Questa verità di riferimento non è però infinitamente precisa: i metadati del foglio dichiarano un errore medio di ricampionamento di 0.56 m (massimo 1.28 m, su 76 punti di taratura). È un **pavimento**: un errore misurato sotto mezzo metro non descrive più la qualità dell'algoritmo, descrive l'incertezza del riferimento stesso, e va letto con questa avvertenza in tutte le tabelle del capitolo 5.

La correttezza della composizione di H_true è verificata da un test: un punto trasformato avanti e indietro torna su sé stesso entro $1.1 \cdot 10^{-13} px$, contro la soglia dichiarata di $10^{-9} px$.

4. Pulizia delle immagini e ricerca delle corrispondenze

Perché pulire l'immagine. Su una fotografia normale una soglia unica potrebbe separare lo sfondo dal soggetto; su un foglio d'archivio no, perché il colore della carta cambia da una zona all'altra del foglio (ingiallimento, pieghe, macchie). Il progetto confronta tre modi di preparare l'immagine prima della ricerca dei punti corrispondenti:

- **Otsu:** una soglia unica per tutta l'immagine, calcolata automaticamente. È il metodo di riferimento più semplice, ed è messo in griglia proprio per avere un termine di paragone.
- **Sauvola:** una soglia diversa per ogni zona del foglio, ricalcolata localmente. Si adatta a un fondo che cambia luminosità, al prezzo di un calcolo più oneroso (reso comunque rapido con una tecnica di somme precalcolate).
- **CLAHE:** non separa l'immagine in bianco e nero, ma ne aumenta il contrasto zona per zona. Serve a verificare l'ipotesi che ridurre l'immagine a due soli livelli distrugga proprio le sfumature su cui i descrittori si basano.



Contrariamente all'ipotesi di partenza, sui ritagli reali **Otsu non fallisce**: il foglio è illuminato in modo abbastanza uniforme da non mettere in crisi una soglia globale. Messo alla prova con un gradiente di illuminazione sintetico e crescente, però, Otsu classifica come inchiostro fino a metà dell'immagine (contro carta), mentre Sauvola resta stabile: il limite teorico di Otsu è reale, semplicemente questo foglio non lo mette alla prova. Riportare una previsione smentita dai dati, invece di adattare l'esperimento per confermarla, è parte del criterio di onestà con cui è condotto il progetto.

Trovare e abbinare i punti. SIFT individua i punti "distintivi" cercando i massimi locali di un filtro sensibile alle macchie (blob), non ai bordi; li descrive con un vettore di 128 numeri costruito dai gradienti locali, ed è per progetto invariante a scala e rotazione. Un abbinamento si accetta solo se il candidato migliore batte nettamente il secondo migliore (il *ratio test* di Lowe) — un filtro severo, che su un reticolo di linee sottili quasi privo di macchie lascia poche centinaia di candidati. **ORB** persegue lo stesso obiettivo in modo più economico — un descrittore binario invece di 128 numeri — e filtra gli abbinamenti in modo meno severo (basta l'accordo reciproco fra i due punti), lasciandone passare molti di più.

Scegliere quanto la trasformazione può deformare. RANSAC cerca la trasformazione migliore all'interno di una famiglia scelta a priori: una **similarità** (può spostare, ruotare, ingrandire), un'**affine** (in più può stirare e inclinare) o un'**omografia** (in più può simulare una prospettiva). Più libertà sembra sempre meglio, ma non lo è quando la maggior parte degli abbinamenti è sbagliata: con più parametri liberi diventa più facile trovare un consenso anche fra dati casuali. Il capitolo 5 mostra che su questo problema è proprio questa scelta, più del matcher usato, a decidere fra successo e fallimento.

5. Risultati sperimentali

I risultati sono organizzati in tre esperimenti crescenti in difficoltà: un caso sintetico per verificare che il programma sia corretto, il caso reale per misurarne le prestazioni, e un confronto con un metodo di deep learning.

5.1 Il programma è corretto (esperimento sintetico)

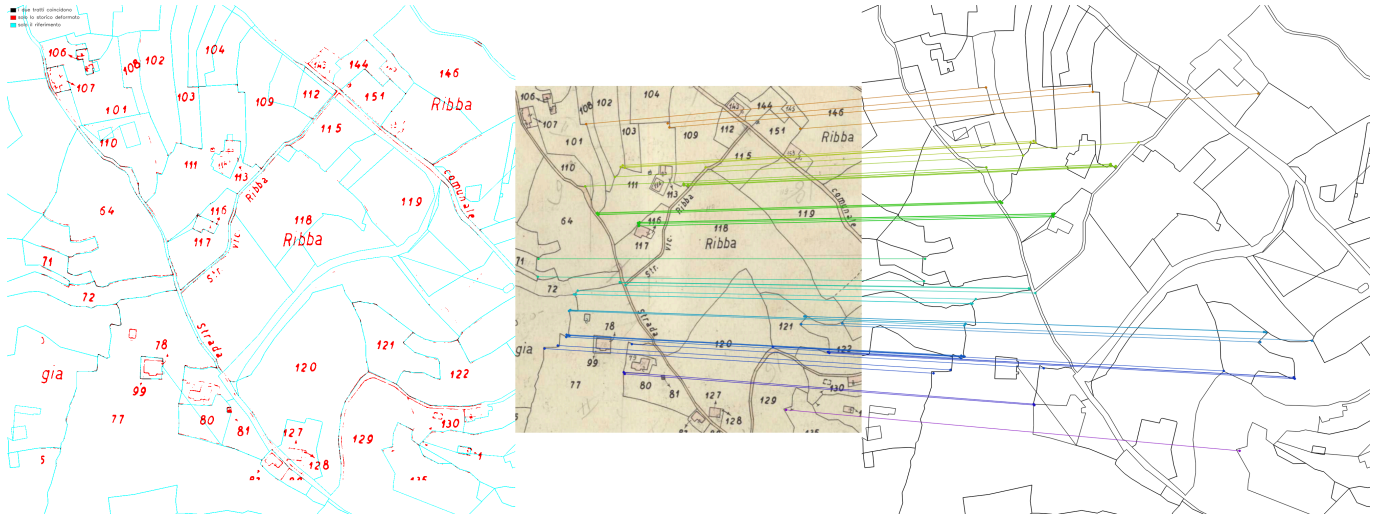
Prima di misurare il caso difficile, occorre verificarne uno facile: ogni ritaglio storico è stato confrontato con **se stesso** trasformato con una rotazione, scala o prospettiva nota. Qui la trasformazione vera è nota per costruzione e non c'è alcun divario fra i due stili di disegno: se il programma sbagliasse anche qui, il problema sarebbe nel codice, non nel compito.

Nel caso peggiore l'errore recuperato resta nell'ordine del pixel — pochi centimetri sul terreno, ben sotto il pavimento di 0.5 m del capitolo 3. Con un'unica eccezione degna di nota, ripresa al punto 5.3: LoFTR perde completamente la trasformazione quando la rotazione supera i 30 gradi, mentre SIFT e ORB restano precisi fino a rotazioni di 90 gradi. La ragione è strutturale — SIFT e ORB calcolano esplicitamente un orientamento di riferimento per ogni punto, LoFTR no, ed è quindi vincolato a ciò che ha incontrato durante l'addestramento su fotografie ordinarie, dove rotazioni così ampie sono rare.

5.2 Il caso reale: storico contro cartografia vettoriale

Il confronto vero è fra ogni ritaglio storico e il raster ottenuto disegnando il file vettoriale sulla stessa zona. Qui la posizione reciproca fra le due immagini è nota solo attraverso i world file, mai vista dall'algoritmo.

Il risultato più importante del progetto è che **la registrazione riesce**, ma con una configurazione controintuitiva: non SIFT ma **ORB**, non l'omografia più flessibile ma la **similarità** più vincolata, con la binarizzazione **Sauvola**. In questa configurazione il 90% delle prove ha successo, con un errore mediano di 0.33 m — sotto il pavimento dell'incertezza del riferimento, cioè al limite di quanto questo confronto può misurare.



Perché ORB batte SIFT è chiarito da un numero solo: il filtro di ORB lascia passare centinaia di abbinamenti in più rispetto al filtro severo di SIFT. Su un problema dove solo l'1-10% degli abbinamenti è corretto, RANSAC ha bisogno soprattutto di **quantità di candidati**, non di candidati già puliti. Si potrebbe pensare che basti rendere meno severo anche il filtro di SIFT (il *ratio test*, §4): è stato verificato, ed è falso.

Allentando la soglia da 0.75 a 0.99 le corrispondenze passano da 168 a oltre 3000, ma l'errore mediano resta a oltre 100 metri e le prove riuscite non aumentano: i candidati aggiuntivi sono rumore puro, non segnale che il filtro teneva nascosto. Il limite di SIFT su questi dati sta nel descrittore, non nel filtro che lo seleziona.

Perché la similarità batte l'omografia si spiega in modo analogo: con inlier ratio così bassi, concedere più libertà geometrica allo stimatore significa solo dargli più modi di accordarsi con dati sbagliati.

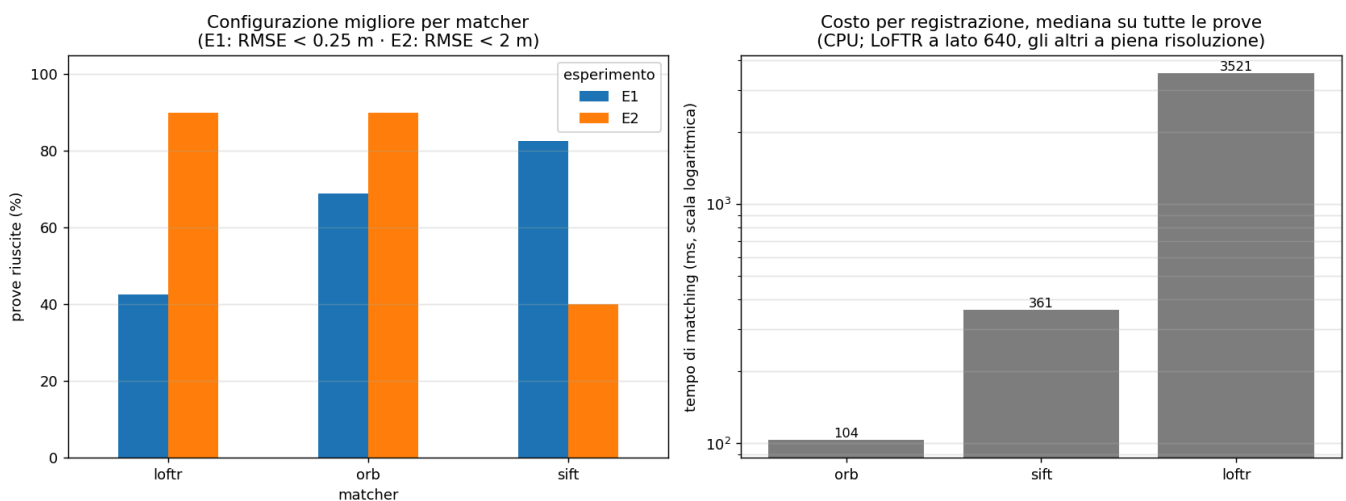
Passando dall'omografia alla similarità il tasso di successo sale dal 27% al 52% a parità di tutto il resto — è, fra i risultati del progetto, quello con la morale più generale: **su dati cross-domain con pochi abbinamenti corretti, il modello geometrico più vincolato non è una semplificazione, è una necessità**. La stessa tabella mostra anche che disegnare nel raster moderno anche acque e strade, non solo i confini di particella, aiuta

(42% di successo contro 35%): sono la parte del vettoriale dove il tratto storico è più marcato.

Va detto, per onestà, che una parte del vettoriale non ha alcuna corrispondenza nell'inchostro storico: il file CXF è la mappa **di oggi**, non una trascrizione dell'impianto, e un secolo di frazionamenti ha introdotto confini che nel 1930 non esistevano. Non è un errore di allineamento, ma va tenuto presente nel leggere le percentuali di corrispondenze corrette, strutturalmente basse.

5.3 Il confronto con il deep learning

L'ultimo esperimento sostituisce, nella stessa identica pipeline, il matcher classico con **LoFTR**, un metodo di deep learning pre-addestrato che non cerca prima i punti distintivi e poi li descrive, ma confronta direttamente le due immagini a griglia. L'idea è che questo approccio possa reggere meglio proprio dove i metodi classici soffrono, cioè dove non ci sono angoli o macchie ben definiti da individuare.



Il risultato è netto ma non nella direzione attesa: **LoFTR pareggia ORB sul tasso di successo, con un errore leggermente peggiore e un tempo di calcolo di un ordine di grandezza superiore**. La promessa del metodo neurale non si realizza su questo tipo di dati. È però nettamente superiore in un aspetto: la qualità delle corrispondenze che produce è pulita fin da subito (fino al 30% di abbinamenti corretti, contro il 2-13% dei metodi classici), a dimostrazione che il collo di bottiglia dell'intero problema è altrove — nel divario fra i due stili grafici, non nella capacità di descrivere un punto.

Anche LoFTR, inoltre, **funziona solo se l'immagine è binarizzata**: con CLAHE non trova quasi nessuna corrispondenza utile e non registra mai; con Sauvola arriva al 90% di successo, esattamente come ORB. Il vantaggio del pre-addestramento su milioni di fotografie non basta da solo a colmare il divario di dominio: a farlo è, ancora una volta, il preprocessing.

6. Limiti dei risultati

Tre avvertenze, dichiarate qui perché condizionano come leggere ogni numero del capitolo precedente.

Il vettoriale non è la digitalizzazione dello storico. È la cartografia in vigore oggi, aggiornata attraverso un secolo di frazionamenti: alcune linee del vettoriale non hanno alcuna controparte nell'inchiostro dell'impianto, non per errore ma per la storia della mappa stessa. È una delle ragioni per cui le percentuali di corrispondenze corrette restano basse anche nelle configurazioni che registrano bene.

Il dataset è di validazione, non di applicazione diretta. L'Agenzia delle Entrate fornisce, per questo foglio, una posizione già nota — ed è proprio questo a rendere possibile misurare l'errore in metri. Su un archivio storico reale, senza world file né riferimento vettoriale, non si potrebbe applicare lo stesso criterio di verifica: resterebbe l'ispezione visiva, più lenta e meno conclusiva. Le decisioni validate qui — modello geometrico vincolato, binarizzazione locale, diffidenza verso le metriche di allineamento indirette — restano valide altrove; il modo di verificarle in metri, no.

Le misure indirette di allineamento non sono affidabili su questi dati.

Un primo tentativo di verificare l'allineamento con la correlazione fra immagini, prima di adottare la verità di riferimento del capitolo 3, ha prodotto risultati che sembravano coerenti ma erano falsi: su un disegno al tratto, righe tracciate a caso ottengono un punteggio di correlazione già alto quasi quanto il vettoriale vero, perché l'inchiostro è sparso ovunque sul foglio. Da qui la regola seguita in tutto il progetto: l'unica misura accettata è l'errore in metri contro la verità di riferimento esatta, mai una misura indiretta senza un confronto con un caso casuale.

7. Uso pratico e conclusioni

Lo stack tecnologico. Il nucleo classico è scritto in Python 3.12 con OpenCV (SIFT, ORB, RANSAC, deformazione dell'immagine), NumPy, pandas, Matplotlib e Pillow — senza librerie geospaziali come `pyproj` o `geopandas`: raster e vettoriale sono già nello stesso sistema di riferimento, quindi non c'è nulla da riproiettare. Il solo componente B (LoFTR) richiede `torch` e `kornia`, caricati solo se il matcher scelto è `loftr`: senza quella scelta la pipeline classica gira anche su una macchina senza le due librerie installate. I pesi del modello (~90 MB) sono distribuiti insieme al progetto, non scaricati al primo avvio.

7.1 Installazione

```
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Per il solo approccio classico bastano **numpy**, **opencv-python**, **Pillow**, **scipy**, **pandas**, **matplotlib**.

torch e **kornia** servono **solo** a **--matcher loftr**; su CPU conviene l'indice dedicato:

```
pip install torch==2.3.1 --index-url https://download.pytorch.org/whl/cpu
pip install kornia==0.7.3
python -m scripts.scarga_pesì           # pesi LoFTR + verifica del checksum
```

I dati cartografici non sono versionati: [data/README.md](#) documenta come ricostruirli.

7.2 La CLI

```
python -m src.main --hist <crop.png> --modern <raster.png> [opzioni]

--hist <path>           immagine storica (obbligatorio)
--modern <path>         immagine di riferimento (obbligatorio)
--matcher <sift|orb|loftr>      default: sift
--preprocess <none|clahe|otsu|sauvola> default: sauvola
--morph-close <n>        default 0
--morph-open <n>         default 0
--model <similarity|affine|homography> default: homography
--ratio <float>          ratio test di Lowe, default 0.75
--ransac-thresh <float>   px, default 3.0
--seed <int>             default 42
--jgw-hist <path>        world file storico   (solo evaluate)
--jgw-modern <path>     world file moderno   (solo evaluate)
--soglia-m <float>      soglia di successo in metri
--out-csv <path>       default results/runs.csv (in append)
--out-figure <path>    overlay del warp + corrispondenze
--esperimento <str>    etichetta nella colonna `esperimento` del CSV
--crop <str>           etichetta nella colonna `crop` del CSV
--verbose              descrizione dettagliata delle operazioni
--help                questa lista di comandi
```

Esempio 1 — la configurazione migliore su dati reali:

```
python -m src.main \
  --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher orb --preprocess sauvola --model similarity \
  --jgw-hist data/crops/ribba.jgw --jgw-modern data/crops/ribba_vec.jgw \
  --soglia-m 2.0 --out-figure results/figures/registrazione.png
```

Esempio 2 — senza world file: la pipeline gira lo stesso. È la prova che la georeferenziazione non entra nell'algoritmo:

```
python -m src.main --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher sift --preprocess sauvola
# → H_est prodotta, "nessun world file: RMSE non calcolato"
```

Esempio 3 — lo stesso confronto con il matcher neurale, cambiando una parola:

```
python -m src.main --hist data/crops/ribba.png --modern data/crops/ribba_vec.png \
  --matcher loftr --preprocess sauvola --model similarity
```

7.3 Preparazione dei dati ed esperimenti

```
# ritagli dal foglio (PNG + world file affiancato)
python -m src.prep.crop

# ispezione e verifica del parser CXF
python -m src.io_geo

# rasterizzazione del vettoriale sulla zona di un ritaglio
python -m src.prep.rasterize --crop ribba --codici 18,12
python -m experiments.m7_rasterize_check --crop ribba

# confronto dei preprocessing
python -m experiments.m5_preprocess --crop tutti --dettaglio ribba

# gli esperimenti: E1, E2, la diagnosi del ratio, E3
python -m experiments.m6_e1_completo --riparti
python -m experiments.m8_e2_griglia
python -m experiments.m8_e2_griglia --diagnosi-ratio
python -m experiments.m9_e3_loftr

# figure e tabelle, tutte generate dal CSV
python -m src.report --csv results/runs.csv
```

7.4 Riprodurre tutto in un comando

I comandi di §7.3 vanno eseguiti in quest'ordine, e l'ordine non è arbitrario:

i ritagli prima della rasterizzazione, la rasterizzazione prima di E2, gli esperimenti prima delle tabelle, le tabelle prima di questo documento. Eseguirli a mano funziona, ma un passo dimenticato non dà errore: produce un CSV parziale e tabelle che sembrano complete.

```
python -m scripts.riproduci --controlla # verifica le precondizioni, non esegue
python -m scripts.riproduci --lista      # le fasi, in ordine, con i tempi
python -m scripts.riproduci              # tutto tranne E3 (~18 min)
python -m scripts.riproduci --con-loftr  # tutto, E3 compreso (~60 min)
python -m scripts.riproduci --da-verifica-raster # senza i dati AdE, dai soli
ritagli
```

Lo script stampa ogni comando prima di eseguirlo — il log di una corsa è la versione eseguita di §7.3 — e dopo ogni fase verifica che gli artefatti attesi esistano davvero: un comando che esce con codice 0 senza aver scritto quello che doveva ferma la corsa, che riprende con `--da <fase>`. In coda controlla che ogni figura citata da questa relazione sia stata prodotta e che nessun segnaposto di tabella sia rimasto vuoto.

Il controllo delle precondizioni non è formalità. Gli esperimenti di E1 ed E3 convertono l'errore in metri leggendo la risoluzione da un world file, e se nessuno è disponibile l'errore in metri resta indefinito per ogni riga: l'esperimento gira fino in fondo e conclude "0 riuscite". Si legge come un algoritmo che fallisce, ed è invece un file assente, e la difesa è verificare prima.

Il controllo è però **per fase**, e la distinzione conta proprio alla consegna. Le scansioni catastali non sono ridistribuibili, i ritagli sì: chi riceve il progetto senza `data/raw/` ha comunque in `data/crops/` i ritagli, i raster del vettoriale e i rispettivi world file. Solo `crop`, `cxr` e `rasterize` aprono le scansioni; le altre nove fasi no.

In sintesi:

1. La pipeline è verificata corretta sul caso sintetico, con errore sub-pixel.
2. Sul caso reale la registrazione riesce (90% delle prove, errore mediano 0.33 m), ma con la combinazione meno prevedibile: descrittore economico (ORB), modello geometrico vincolato (similarità), binarizzazione locale (Sauvola).
3. Il fattore più determinante non è il matcher scelto, ma **quanta libertà si concede al modello geometrico**: con pochi abbinamenti corretti, vincolare è necessario, non un compromesso.
4. Un matcher di deep learning pre-addestrato (LoFTR) non supera i metodi classici su questo compito: pareggia il migliore di essi, a un costo di calcolo molto maggiore, e produce corrispondenze più pulite senza che questo si traduca in un risultato finale migliore.
5. Il preprocessing conta più della scelta dell'algoritmo di matching, classico o neurale: è la binarizzazione locale a rendere il problema trattabile per tutti e tre i metodi messi alla prova.

Per concludere il progetto misura, e spiega, dove i metodi classici cedono sul confronto fra stili grafici così diversi — e dove reggono meglio del previsto. Quello che vale oltre questo foglio non sono le cifre in metri, legate a questa scala e a questo vettoriale, ma le decisioni che le hanno prodotte: vincolare il modello geometrico quando i dati sono rumorosi, ripulire il segnale con una soglia locale, e non fidarsi di una misura indiretta che non batta nettamente un caso casuale.